AI Ingestion Pipeline — Technical Architecture

GraphOne / FrontierAtlas — AI Engineer Demo Task

1. Architecture Overview

The pipeline follows a modular architecture:

Sources → Async Crawlers → Normalization & Freshness → Entity Resolution → Enrichment → Storage

Python `asyncio` and `aiohttp` are used for asynchronous HTTP ingestion. Each data vertical has an independent crawler, while the orchestrator coordinates collection and CSV export. This modular design allows individual sources to scale independently.

2. Scale Strategy — 500K+ Records

For production scale, crawler workloads would be divided into independent partitions such as source, category, and page ranges. A durable queue would distribute these partitions across multiple asynchronous workers.

Workers would use bounded concurrency, per-domain rate limits, checkpoints, and batch writes. Checkpoints allow interrupted jobs to resume without restarting the complete crawl.

The architecture can therefore scale from the current demo to hundreds of thousands of records primarily by increasing worker and infrastructure capacity.

3. Async Crawling, Retries & Rate Limits

The crawler uses asynchronous HTTP requests through `aiohttp`. Request timeouts prevent stalled connections from blocking workers.

HTTP `429 Too Many Requests` responses are handled as transient failures using retry and increasing backoff. In production, per-domain concurrency limits, jitter, and token-bucket rate limiting would prevent synchronized requests.

Individual source failures are isolated so that one unavailable source does not terminate the complete ingestion pipeline.

4. 413 Payload Protection & LLM Extraction

For production LLM extraction, raw pages would first be cleaned of unnecessary HTML and boilerplate. Content size would then be estimated before sending it to an LLM.

Oversized content would be split into semantically meaningful chunks while preserving important metadata such as title, source URL, and publication date.

A multi-provider fallback strategy can use:

Gemini Flash → Groq Llama → DeepSeek

Provider-specific rate limits and exponential backoff would prevent repeated failures. A bounded retry budget prevents a failed provider from blocking the complete pipeline.

5. Freshness & Duplicate Prevention

News and job records are normalized to ISO-8601 timestamps and filtered against the required 24-hour freshness window.

For distributed workers, each record should receive a deterministic fingerprint based on its source URL, canonical URL/title, and normalized publication timestamp.

A unique database constraint or distributed key-value store can then prevent duplicate processing across crawler nodes.

6. Deterministic Entity Resolution

The entity resolver normalizes names by removing case differences, punctuation, whitespace variations, and common corporate suffixes such as:

`Inc`, `Corp`, `Ltd`, `LLC`

Normalized names are then mapped against a canonical entity dictionary.

The pipeline also produces an **Entity Mapping Log** containing:

- `raw_name`
- `canonical_name`

This provides an auditable record of how extracted entity names were resolved.

7. Storage Strategy

PostgreSQL would be the primary production database because it provides strong consistency, indexing, transactions, and uniqueness constraints.

Raw HTML/JSON snapshots can be stored in object storage such as S3.

For semantic search, embeddings can be stored using **pgvector** or a dedicated vector database.

For complex relationships such as:

Startup → Product → Research Paper → GitHub Repository

a graph database such as **Neo4j** can be used.

CSV and Google Sheets are treated as presentation/demo outputs rather than the production source of truth.

8. Anti-Bot & JavaScript-Rendered Sources

Sources requiring JavaScript rendering can be isolated behind asynchronous Playwright workers where permitted.

Browser workers should use bounded concurrency, caching, checkpoints, and appropriate request timing.

The system should respect robots.txt, terms of service, and source rate limits. If a source cannot be accessed reliably, the pipeline should record the failure or use an approved alternative source rather than generating fabricated records.

9. Production Deployment

A production deployment can consist of:

Scheduler → Durable Queue → Async Crawler Workers → Enrichment Workers → PostgreSQL/Object Storage

Redis can be used for short-lived coordination and rate-limit state.

Monitoring should track:

- Records discovered
- Records accepted/rejected
- Duplicate rate

- Freshness lag
- HTTP status codes
- Retry counts
- Provider latency
- Source failures

Structured logs and source-level dashboards make the system observable and maintainable.

10. Current Demo Results

Vertical	Result
Startups	1,000
Research Papers	1,001
Jobs	136
News	21
Products	62
Entity Mapping	998 unique names
Automated Tests	9 passed

Integrity Note: All records are intended to originate from legitimate source URLs. The architecture distinguishes implemented demo functionality from production-scale design requirements.